A place for every willing hand.

A volunteer coordination service for fire support

FireLink　Final design review

邓雪意 24168119　朱禹杭 24168202

2026.11



FireLink　Organizer workspace

Make volunteer support actionable

FireLink establishes a trusted connection, then hands coordination to the organization's existing channels.

Describe the need

Organizers specify the work, schedule, location, capabilities and required information.

Make an informed match

Volunteers share task-specific information; organizers review each application.

Connect with consent

After acceptance, authorized members can access only the contact methods the volunteer shared.

Five design priorities from the course brief

Starting with wildfire support, we focus on connecting affected communities and civilian volunteers.

Brief requirement	Our design response
Explain the need	Why do willingness and availability still need structured tasks and human review?
Show a clear user journey	When do volunteers and organizers need information from one another?
Explain flows and design decisions	How do publishing, consent, review and handoff connect?
Ease of use and reliability	How can we reduce ambiguity, mistakes and duplicate work under pressure?
English and UX Manual	Clients support common Chinese and English locale variants; usability principles translated into concrete interactions.

Course brief pp.16–19, 30, 33–34, 44, 47

Three information gaps from sign–up to confirmation

These hypotheses come from the course scenario and role–based analysis; they require user validation.

Is the request specific?

“Help needed” does not tell a person whether they fit. Work, time, place and capabilities must be clear.

Can an application be assessed?

Different tasks need different evidence. Repeated questions and fragmented information hinder review.

Is the outcome clear?

Applying is not confirmation. Decisions, withdrawals and unconfirmed applications need explicit states.

Our scope ends at review and consented contact

FireLink handles publication, applications, review and consented contact; existing channels handle subsequent coordination.

Within FireLink

Publish specific support tasks

Submit task-specific consent

Review people and manage capacity

Notify outcomes and release permitted contacts

Handoff to existing channels

Phone, WeChat and other established channels

The organization confirms people and arrangements

Existing field coordination practices continue

Acceptance and consented contact create a clear handoff into the organization's familiar working practices.

One support task, three responsibilities

Actor	Decision to make	System support
Volunteer	Does this task fit me? What am I willing to share?	Task details, reusable profile, granular consent, application history
Organizer / reviewer	Is the need clear? Does this person fit? Who may contact them?	Task authoring, scoped roles, continuous review, outcomes and notices
Community / request owner	What support is needed, and who is responsible for coordination?	Needs become specific tasks under a shared incident

Organizations remain responsible for decisions and coordination; the platform structures information and records actions.

Course brief pp.16–19; shared/organizer.ts; organizer/src/app.ts

Make the application journey easy to follow

Walkthrough: a simulated wildfire-edge support effort recruiting for supplies, transport and registration.

Stage	Discover	Assess	Apply	Wait	Receive
Volunteer	Browse local incidents	Check time, skills, location	Choose and consent to data	Track or withdraw	Join existing channels if accepted
Question	Where can I help?	Am I a good fit?	What will be shared?	Has a decision been made?	Who do I contact next?
Response	Tasks grouped by incident	Structured task details	Granular consent	Visible state and withdrawal	Outcome and contact handoff

Connect the service journey to backend rules

Service layer	Publish	Apply	Review	Handoff
Volunteer-facing	Incident and task details	Profile, required fields, consent	Pending / withdrawal	Outcome, response, contact route
Organizer actions	Set work, capacity and times	Specify required information	Inspect, decide, respond	Continue in established channels
Backstage	Organization and publish checks	Save consent snapshot	Check access, capacity and state	Notify, disclose by consent, audit

Service evidence: task description → consent record → review outcome → contact handoff.

server/src/organizer-api.ts; server/src/domain.ts; server/test/organizer.test.ts

Two interfaces meet at one application

Organizer	Home / incidents	Create task	Edit & publish	Review	Accept & notify
Volunteer	Discover	Task details	Consent & apply	Application	Outcome & contact

Incomplete information: complete before submitting

Pending: may withdraw; declined: retain the outcome

Still pending when the task ends: expire and notify

After the contact handoff, both parties continue in the organization’s existing channels.

Page flow: organizer/src/app.ts; presentation/proposal/scenes.ts; native client views

Organize the workspace around tasks

Destination	Question it answers	Key actions
Home	What needs attention now?	Filter incidents and prioritize tasks
Tasks	What are we recruiting for?	Edit, publish, pause, close, archive
Applications	Who fits this task?	Inspect requirements, data and respond
Incidents	Which tasks share a context?	Maintain incident context and linked tasks
Notices / organization	What was communicated? Who has access?	Inspect notices, roles, contacts and members

Start with the tasks that need attention



Drill interface Select to view full capture

Focus the context

Filter the incidents relevant to the current work.

Prioritize the work

Sort by updates, pending applicants or time to deadline.

Act from the overview

Resume a draft or open the applications for a task.

Multiple tasks can belong to one incident



Drill interface Select to view full capture

Shared context

One incident can contain multiple specific support tasks.

Location context

An incident location makes the setting understandable.

Separate responsibilities

Capacity, requirements, schedules and owners belong to tasks.

Turn a request into an assessable task



Drill interface Select to view full capture

Write and preview

Group task information and preview what volunteers will read.

Task-specific questions

Add availability and experience questions alongside reusable profile data.

Reliable saving

Drafts autosave. Requirements lock once applications exist.

Understand the work before applying



Drill interface Select to view full capture

Understand the work

Count, sort and pack supplies at an agreed support point.

Check the arrangement

Distinguish start, end, application deadline and meeting point.

Know the boundaries

The drill describes support boundaries and asks volunteers to wait for confirmation.

Review applications against task requirements



Drill interface Select to view full capture

Stay in the review flow

Keep the queue and detail together, with previous and next navigation.

Decide with context

Compare task requirements with the shared profile and submitted answers on availability and experience.

Separate audiences

The response goes to the volunteer; internal notes stay with the organization.

Assess the match before sharing contacts

Consent belongs to an application; contact access also requires role authorization.

Information / state	Pending	Accepted + permitted role	Declined / withdrawn / expired
Shared profile and task answers	Used for task-scoped review	Retained as application context	Retained with the outcome
Consented contact methods	Hidden	Only consented methods exposed	Hidden
Internal notes	Organization only	Organization only	Organization only
Volunteer receives	Submission state and later outcome	Decision and public response	Status and outcome explanation

Once applications exist, profile, contact and custom-question requirements lock together.

Separate deadlines, task endings and outcomes

Task state and application state are related, but distinct.

Application deadline

Stop new applications; existing applications can still be reviewed.

Task end

Pending applications expire and receive an outcome notification.

Explicit decision

Accepted, declined and withdrawn remain distinct; the server checks the latest state.

Server-side capacity and concurrency checks keep the displayed decision consistent with the stored outcome.

Show tasks, consent and outcomes to volunteers



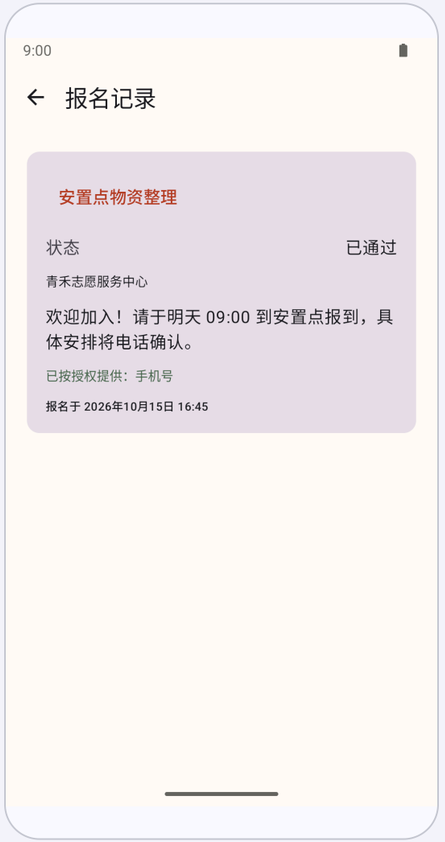
Understand the task

Read the organizer, requirements and details.



Choose what to share

Distinguish required and optional information.



Receive a clear outcome

Move to existing contact channels after acceptance.

A deliberate handoff to familiar channels



Drill interface Select to view full capture

Keep an outcome record

Organizers inspect sent notices; volunteers can revisit outcomes.

Release contacts by consent

Authorized members establish contact after acceptance.

Fit existing practice

The organization confirms subsequent arrangements and changes through existing channels.

Consistency makes the workspace easier to read

A restrained blue palette

Primary actions, selections and focus share a coherent language.

More than color

Labels, outlines and selection indicators reinforce state.

One interaction vocabulary

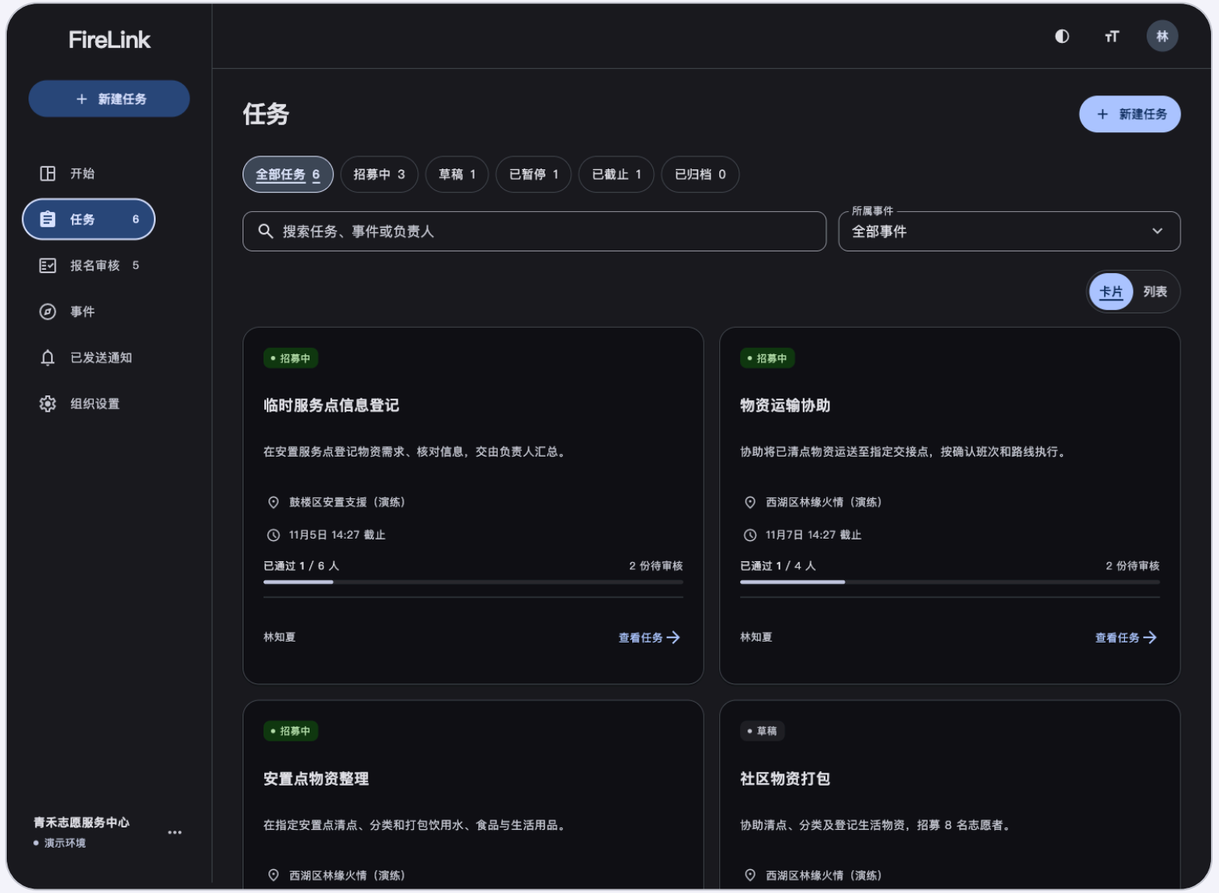
Reuse controls and feedback; support light, dark and larger text.

Primary
#445E91

Selected
#D8E2FF

Surface
#F3F3FA

Text
#1A1B20



Drill interface Select to view full capture

Improve the interface through targeted reviews

Issue in review records	Current response	How to judge value
Scattered information and unclear next action	Task-focused home and continuous queue/detail review	Can the reviewer find the next relevant application?
Narrow-screen and large-text pressure	Shared control sizing, reflow and focused mobile detail	Inspect real copy, keyboard flow and enlarged text
Inconsistent themes, states and components	Semantic theme, shared components, stable status colors	Targeted contrast and component-state checks

Three interfaces, one set of business rules

Interfaces	Organizer web workspace Android client Apple clients
Service & contracts	Shared APIs, task content, consent fields and application states
Data & constraints	Cloudflare Workers D1 R2; access, capacity, state and records

One set of rules connects publishing, consent, review and outcomes across both interfaces.

Automated checks for critical rules

49

Relevant backend tests passed

4

Color scheme tests passed

Re-run for this review

Covers access, consent, capacity and state rules, including custom fields and answer snapshots; provides dark mode support and automatic color scheme generation.

Rules enforced on the server

Acceptance after withdrawal, competition for the final slot and contact disclosure each have explicit expected outcomes.

Reusable verification

Retained scenarios and test entrypoints make core behaviors checkable with each iteration.

Local run 2026-11-04: server/test/{organizer,api}.test.ts; organizer/test/theme.test.ts

Next: validate the flow and task descriptions

First observe whether organizers and volunteers can complete a simulated civilian support task.

Validate the service

Ask organizers and volunteers to complete the connected flow; observe misunderstandings, wrong decisions and unclear handoffs.

Improve task authoring

Develop templates and field guidance for common support tasks to reduce authoring and review effort.

Explore organizational adoption

Pilot with volunteer organizations, communities and training partners; assess maintenance and support costs.

Support starts with a shared understanding.

Clear needs Informed decisions Consented connections

For volunteers

Know whether the task fits and how information is used.

For organizers

Use one flow to express needs, confirm people and hand off contact.

FireLink

Appendix | Mapping the brief to outcomes

Requirement / source	Deck evidence	Qualification
Topic 2.4 p.30	Civilian volunteer wildfire–support service	Support recruitment, matching and handoff
Human–centered, practical constraints p.17	Roles, journey and blueprint	Role–based task analysis and target journey
Journey and page flow p.44	Main slides 7—9	Main path and exception branches
Design highlights p.44	Main slides 11—20	Implemented interfaces and rationale
UX Manual p.44	Controls, state, feedback, contrast, permissions	Apply the usability and reliability principles in the course brief
English / within 20 min pp.44, 47	Bilingual deck, 26 main slides and appendix	Existing product screenshots remain in Chinese

Appendix | Two connected lifecycles

Object	States	Meaning and constraints
Task	Draft → open	Validate publishing requirements, organization status and contacts
Task	Open ↔ paused	Suspend new enrollment and resume when appropriate
Task	Closed → archived	Close recruitment and retain task history and outcomes
Application	Pending → accepted / declined	Human review, capacity check and result notification
Application	Pending → withdrawn	A stale review cannot override withdrawal
Application	Pending → expired	Triggered by task end; explicit prior decisions remain intact

shared/organizer.ts; server/test/organizer.test.ts

Appendix | Give each data field a purpose

Data	Purpose	Access boundary
Schedule, location, capacity	Support fit and planning	Validate publishing; make updates explicit
Profile, equipment, vehicles, evidence	Assess task-specific suitability	Task-specific consent, reviewed by the organizer against the task
Phone, WeChat, email	Handoff after acceptance	Acceptance + authorized role + consented methods
Volunteer response	Explain the decision	Visible to the volunteer with the outcome
Internal notes and activity records	Internal collaboration and traceability	Separate audiences; record access and disclosure

Appendix | Exceptions need explicit outcomes

Situation	Current behavior	Why it matters
Concurrent decisions for the last slot	Server checks capacity and state	Avoid over-acceptance
Stale review after withdrawal	Reject stale transitions	Respect the volunteer's withdrawal
Pending at task end	Expire and notify once	End indefinite waiting
Concurrent task or incident edits	Version checks and conflict feedback	Avoid silent overwrites
Disconnection or submission timeout	Show connectivity state; verify outcome before retry	Reduce duplicates and uncertainty
Arrangements change after acceptance	Organization confirms through existing channels	Preserve the intended handoff

server/test/organizer.test.ts; organizer/src/api.ts; team-confirmed scope

Appendix | A practical usability study

Plan: invite 3—5 organizers and 5—8 volunteers to evaluate the flow using shared drill tasks.

Study task	Observe / record	Question to answer
Publish a supplies task	Missing information, help requests, task time	Can an organizer author a usable request?
Find a task and apply	Comprehension, required-field errors, consent recall	Does the volunteer understand fit and data sharing?
Review fitting and unsuitable applications	Wrong decisions, navigation loops, missed criteria	Are decisions based on relevant information?
Interpret the result and next step	Correct contact and channel identification	Is the handoff understood?

Appendix | Adoption and the Dräger context

Question	Proposed direction	To validate
Who adopts?	Volunteer groups, community organizations and support coordinators	Ownership of publishing, review and handoff
How does it fit?	Start with one drill or recurring support task	Fit with existing workflows and effort
Why relevant to Dräger?	Respond to the brief's safety, usability and community focus; explore task guidance and training links	Brand requirements, partner interest, content ownership
How is it sustained?	Explore institutional service, training collaboration or public–program support	Costs, responsibilities and funding; no pricing or revenue conclusion

Appendix | Where the implementation lives

Deliverable	Repository evidence	What can be checked
Organizer workspace	organizer/	Tasks, incidents, review, notices, organization and themes
Volunteer clients	android/ apple/	Discovery, tasks, profile, consent, applications and notices
Backend and data	server/src/ server/migrations/	Access, capacity, state, storage and notifications
Shared models and API	shared/ contracts/openapi.json	Task, content-block, map and service contracts
Verification	server/test/ organizer/test/	Rules, themes, browser and accessibility checks
Final presentation	presentation/final/	Bilingual slides, static captures, notes and offline edition

Appendix | Four design decisions

How does it complement group chat?	Chat works well for ongoing coordination. FireLink structures tasks, consent and decisions before handing people to established channels.
What does human review contribute?	Context, evidence and organizational responsibility require human judgment. The system organizes evidence and enforces state rules.
Why delay contact access?	Assess fit first and avoid unnecessary disclosure. Acceptance still requires consent and role-based access.
Why hand off after acceptance?	Organizations already have communication and field practices. We focus on recruitment, matching and trusted handoff to ease adoption.

Appendix | Sources and verification

Original course material	1008–BJUT+DRAEGER.pdf, 48 pages. Key sections: pp.16—19 context; p.30 topic; pp.33—34 usability and reliability; p.44 UI criteria; p.47 schedule.
Product evidence	Current source, component inventory, proposal scenes and native screenshots. Organizer captures from an isolated local drill workspace on 2026–11–04.
Verification for this deck	49/49 relevant backend tests and 4/4 color scheme tests passed. Existing reviews inform component, interaction and responsive–layout iteration.
Drills and analysis	Role–based task analysis, target journey and service blueprint. Synthetic incidents, organizations and people demonstrate the flow in a working prototype.